

CStats - Lab #9: Convolutional Neural Networks

May 5, 2026

Exercise 1.

A grayscale image can be treated as a matrix of numbers. For example, the following 4×4 image has a bright 2×2 square in the middle:

$$X = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

1. How many input numbers does this image contain?
2. A color image of size 100×100 has 3 channels: red, green, and blue. How many input numbers does it contain?
3. Suppose we feed that $100 \times 100 \times 3$ image into a fully connected hidden layer with 1000 neurons. How many weights are in this first layer, ignoring biases?
4. Why might this be a strange first layer for image processing? Give an intuitive answer, not just a parameter count.

Solution 1.

(a) The image has $4 \cdot 4 = 16$ input numbers.

(b) A color image has $100 \cdot 100 \cdot 3 = 30000$ input numbers.

(c) A fully connected layer with 1000 neurons has

$$30000 \cdot 1000 = 30000000$$

weights, before adding the 1000 biases.

(d) This is strange because most useful visual patterns are local. An edge, corner, eye, beak, or texture patch is usually much smaller than the full image. A neuron that immediately looks at every pixel is using many weights to learn something that might only need a small local window. Also, the same small pattern can appear in many locations. A fully connected layer learns separate weights for each location, while a convolutional layer

can reuse the same detector across the image.

Exercise 2.

This exercise introduces the basic convolution operation. Consider the input image

$$X = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

and the 2×2 filter

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

We use stride 1 and no padding, so the filter only sits where it fits completely inside the image. The figure for this exercise is shown at the end of the lab under **Exercise 2**.

1. Compute the top-left output value. That is, place K on the top-left 2×2 window of X and take the entry-by-entry product and sum.
2. Compute the full output feature map.
3. What kind of tiny pattern is this filter looking for?

Solution 2.

(a) The top-left window is

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}.$$

The output is

$$1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 1 \cdot 1 = 2.$$

(b) Sliding the filter one pixel at a time gives

$$\begin{bmatrix} 2 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \end{bmatrix}.$$

The output is 3×3 because a 2×2 filter fits in 3 horizontal positions and 3 vertical positions of a 4×4 image.

(c) This filter rewards a diagonal from top-left to bottom-right. It gives a larger value when the local 2×2 window has bright pixels on that diagonal. In a learned CNN, the entries of the filter are not chosen by hand. They are trainable parameters adjusted by gradient descent.

Exercise 3.

A CNN does two important things: it looks locally, and it reuses the same filter in many locations.

Imagine an 8×8 grayscale image and a detector that only needs to inspect a 3×3 window. The figure for this exercise is shown at the end of the lab under **Exercise 3**.

1. With no padding and stride 1, how many different 3×3 windows are there in the 8×8 image?
2. Suppose we make a locally connected layer with one neuron per 3×3 window, but every neuron has its own separate weights. How many parameters are needed, including one bias per neuron?
3. Suppose instead we use one convolutional filter shared across all locations. How many parameters are needed, including one bias?
4. Suppose we use 64 different shared 3×3 filters. What is the output shape? How many trainable parameters are in this convolutional layer?
5. Explain why parameter sharing is a good bias for image models.

Solution 3.

(a) A 3×3 window fits in $8 - 3 + 1 = 6$ positions horizontally and 6 positions vertically, so there are

$$6 \cdot 6 = 36$$

windows.

(b) Each local neuron has $3 \cdot 3 = 9$ weights and 1 bias, so 10 parameters per neuron. With 36 neurons, this is

$$36 \cdot 10 = 360$$

parameters.

(c) One shared convolutional filter has the same 9 weights no matter where it is applied, plus 1 bias. This gives only

$$9 + 1 = 10$$

parameters.

(d) Each of the 64 filters produces a 6×6 feature map, so the output shape is

$$6 \times 6 \times 64.$$

The number of trainable parameters is

$$64 \cdot (3 \cdot 3 + 1) = 640.$$

(e) The same visual pattern can appear in different places. A bird's beak, a vertical edge, or a wheel should be recognizable whether it is near the top-left or bottom-right. Sharing a filter says: learn one detector, then try it everywhere. This greatly reduces the parameter count and makes the model's first layers better matched to images.

Exercise 4.

Now we use filters as simple pattern detectors. Consider the vertical-line image

$$X_v = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

and two 3×3 filters:

$$K_v = \begin{bmatrix} -1 & 2 & -1 \\ -1 & 2 & -1 \\ -1 & 2 & -1 \end{bmatrix}, \quad K_h = \begin{bmatrix} -1 & -1 & -1 \\ 2 & 2 & 2 \\ -1 & -1 & -1 \end{bmatrix}.$$

The figure for this exercise is shown at the end of the lab under **Exercise 4**.

1. Place each filter on the center 3×3 window of X_v . Compute the two outputs.
2. Which filter responds more strongly to the vertical line?
3. If the input instead had a horizontal line through the center row, what would you expect to happen?
4. In a real CNN, would we usually hand-code K_v and K_h ? Explain.

Solution 4.

(a) The center 3×3 window of X_v is

$$\begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \end{bmatrix}.$$

For the vertical filter,

$$\langle X_{\text{window}}, K_v \rangle = 0(-1) + 1(2) + 0(-1) + 0(-1) + 1(2) + 0(-1) + 0(-1) + 1(2) + 0(-1) = 6.$$

For the horizontal filter,

$$\langle X_{\text{window}}, K_h \rangle = -1 + 2 - 1 = 0.$$

A more expanded calculation gives the same answer: only the middle column of the input window is nonzero, so we add the middle-column entries of the filter.

(b) The vertical filter responds more strongly: it gives 6 instead of 0.

(c) For a horizontal line through the center row, the horizontal filter would respond strongly and the vertical filter would be near zero. The detector is checking whether the local window resembles its template.

(d) Usually no. In modern CNNs, filters are learned from data. Early filters often become edge or color-blob detectors because those are useful for reducing the classification loss, not because we manually inserted them.

Exercise 5.

Stride controls how far the filter jumps each time it moves. Padding adds artificial border pixels, usually zeros. For a square input of width W , a square filter of width F , padding P , and stride S , the spatial output size is

$$\frac{W - F + 2P}{S} + 1,$$

when this is an integer. The figure for this exercise is shown at the end of the lab under **Exercise 5**.

1. A 7×7 image is convolved with one 3×3 filter, no padding, stride 1. What is the output size?
2. Same image and filter, no padding, stride 2. What is the output size? What changed intuitively?
3. Same image and filter, padding 1, stride 1. What is the output size? Why is padding useful?
4. Apply 2×2 max pooling with stride 2 to the feature map

$$A = \begin{bmatrix} 3 & -1 & -3 & -1 \\ -3 & 1 & 0 & -3 \\ -3 & -3 & 0 & 1 \\ 3 & -2 & -2 & -1 \end{bmatrix}.$$

Compute the pooled output.

5. Fill in the shapes of this small CNN:

$28 \times 28 \times 1$
 $\rightarrow \text{Conv}(8 \text{ filters}, 3 \times 3, S = 1, P = 1)$
 $\rightarrow \text{MaxPool}(2 \times 2, S = 2)$
 $\rightarrow \text{Conv}(16 \text{ filters}, 3 \times 3, S = 1, P = 1)$
 $\rightarrow \text{MaxPool}(2 \times 2, S = 2)$
 $\rightarrow \text{Flatten} \rightarrow \text{Dense}(10).$

Solution 5.

(a) The output spatial size is

$$\frac{7 - 3 + 0}{1} + 1 = 5,$$

so the output is $5 \times 5 \times 1$ if there is one filter.

(b) The output spatial size is

$$\frac{7 - 3 + 0}{2} + 1 = 3,$$

so the output is $3 \times 3 \times 1$. Intuitively, stride 2 skips every other filter position. It computes

fewer local dot products, so the feature map is smaller.

(c) The output spatial size is

$$\frac{7 - 3 + 2}{1} + 1 = 7,$$

so the output stays 7×7 . Padding is useful because repeated convolutions without padding keep shrinking the image. Padding lets boundary pixels participate more often and can preserve the spatial size.

(d) Split A into four non-overlapping 2×2 blocks and take each maximum:

$$\begin{bmatrix} 3 & -1 \\ -3 & 1 \end{bmatrix} \mapsto 3, \quad \begin{bmatrix} -3 & -1 \\ 0 & -3 \end{bmatrix} \mapsto 0,$$
$$\begin{bmatrix} -3 & -3 \\ 3 & -2 \end{bmatrix} \mapsto 3, \quad \begin{bmatrix} 0 & 1 \\ -2 & -1 \end{bmatrix} \mapsto 1.$$

Thus the pooled output is

$$\begin{bmatrix} 3 & 0 \\ 3 & 1 \end{bmatrix}.$$

(e) The shapes are

$$\begin{aligned} 28 \times 28 \times 1 &\rightarrow 28 \times 28 \times 8 \rightarrow 14 \times 14 \times 8 \\ &\rightarrow 14 \times 14 \times 16 \rightarrow 7 \times 7 \times 16 \rightarrow 784 \rightarrow 10. \end{aligned}$$

The convolutional layers create feature maps, pooling reduces spatial resolution, flattening turns the final stack of feature maps into a vector, and the dense layer produces 10 class scores.

Exercise 6.

A convolutional layer is usually followed by a nonlinearity. One common choice is ReLU:

$$\text{ReLU}(z) = \max(0, z), \quad H = \text{ReLU}(X * K + b).$$

The convolution creates a feature map of scores, then the nonlinearity is applied to that feature map entry by entry.

- (a) Compute $\text{ReLU}(Z)$ for

$$Z = \begin{bmatrix} 2 & -1 \\ 0 & 3 \end{bmatrix}.$$

- (b) Now see why nonlinearities are not just decoration. Suppose we stack two 1D convolutions with no biases and no nonlinearities. The first filter is $K_1 = [1, 1]$, and the second filter is $K_2 = [1, -1]$. For input $[x_1, x_2, x_3]$, using valid convolutions, the first layer gives

$$h_1 = x_1 + x_2, \quad h_2 = x_2 + x_3.$$

The second layer gives $y = h_1 - h_2$. Simplify y . What single length-3 filter would give the same output directly from $[x_1, x_2, x_3]$?

- (c) Explain in words why deep CNNs insert a nonlinearity such as ReLU after each convolution.

Solution 6.

- (a) ReLU keeps positive numbers and replaces negative numbers by zero:

$$\text{ReLU}(Z) = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}.$$

- (b) Substitute the first-layer outputs into the second layer:

$$y = h_1 - h_2 = (x_1 + x_2) - (x_2 + x_3) = x_1 - x_3.$$

This is exactly the same as applying one length-3 filter directly:

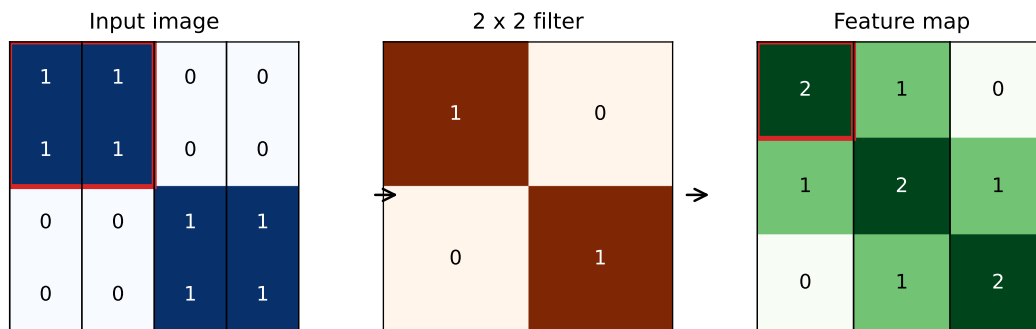
$$[1, 0, -1] \cdot [x_1, x_2, x_3] = x_1 - x_3.$$

So these two linear convolutional layers collapse into one linear convolution with a larger effective filter.

(c) Without nonlinearities, stacking convolutions is still just a linear operation. More precisely, several linear convolutional layers can be combined into one convolutional layer with a larger effective receptive field. ReLU prevents this collapse. It lets the next layer depend on which previous features are active, so the network can build more complex detectors from simpler ones.

Figures

Exercise 2: What is a convolution?



Exercise 3: Local fields and parameter sharing

Same small pattern, two locations

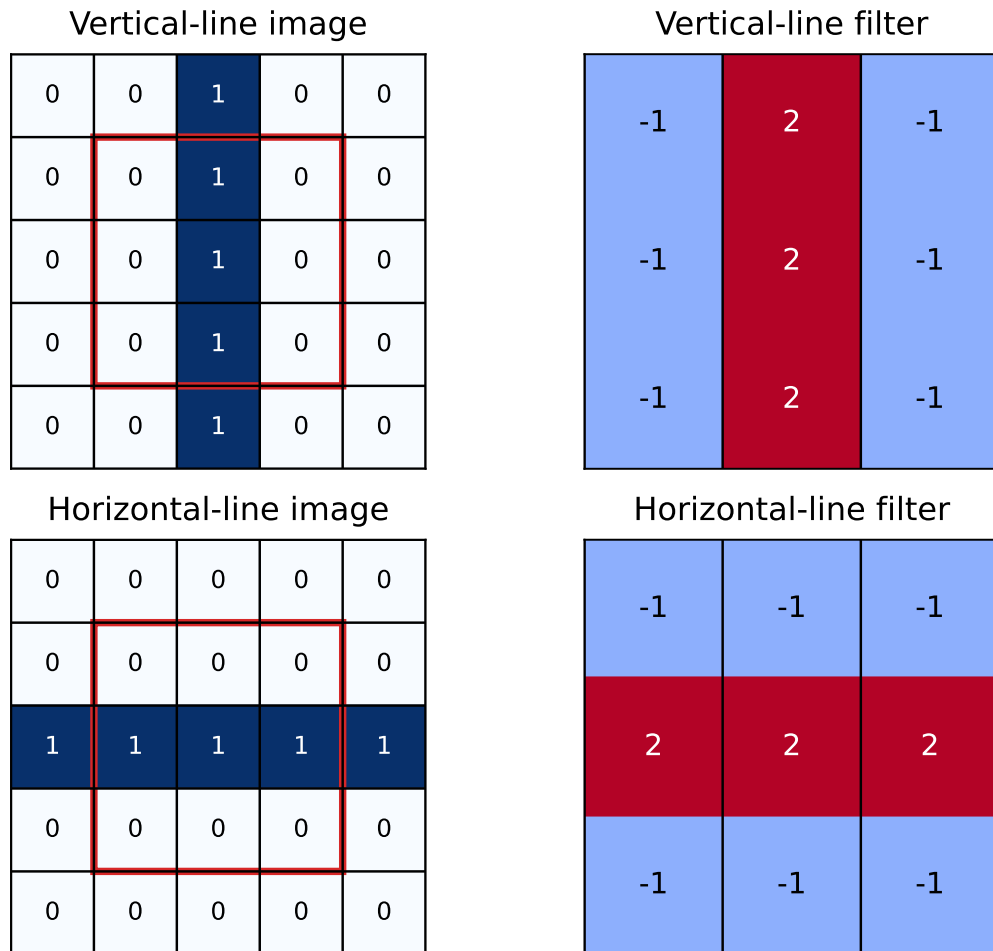
0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	1	1	1	0	0	0	0
0	0	1	0	0	0	0	0
0	0	0	0	0	1	0	0
0	0	0	0	1	1	1	0
0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	0

One shared filter

0	1	0
1	1	1
0	1	0

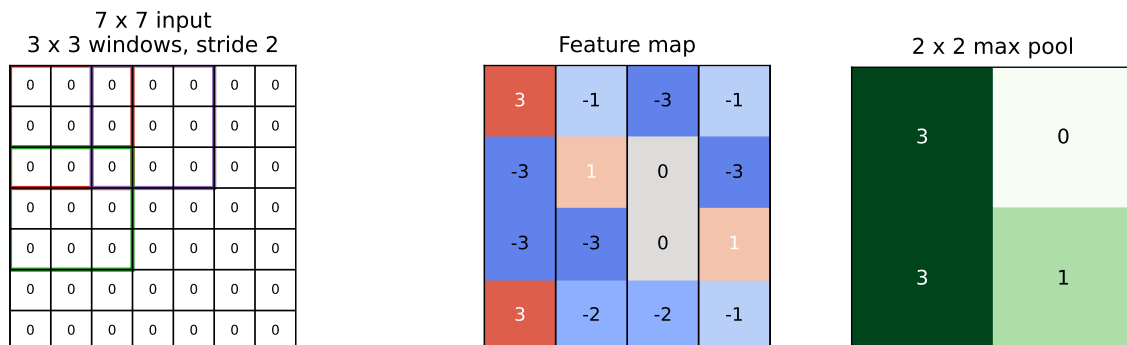
A convolution reuses the same weights at every spatial position.

Exercise 4: Simple filters as pattern detectors



A high dot product means the local window resembles the filter.

Exercise 5: Stride, padding, pooling, and the full CNN



Stride skips positions. Max pooling keeps the strongest response in each small block.